Automated Quoting System Design Summary

This document details the fixed workflow, the enhancements applied to the original proposal, and a strategic comparison of our custom solution against low-code alternatives, focusing on the core project requirements.

1. Revised Automated System Workflow

Step	Action	Tool / Data Persistence
1. Master Data Management (MDM)	HQ staff manually update the single, central Excel file with supplier data. BigQuery database	**Central Excel File** for suppliers, and **RShiny Dashboard** and **BigQuery Connector** for KPIs
2. Customer Data Capture & Trigger	Staff or franchisees input job measurements and customer details via the **R Shiny dashboard**. The staff member clicks a 'Generate Quote' button, triggering the Python script.	**R Shiny Customer Input Dashboard** and the **Execution Environment**.
3. Core Calculation Engine	The **Python Script** simultaneously reads the JSON data from the R Shiny trigger (customer input) and **parses the central Excel sheet** for the latest supplier data. It runs the proprietary formulas.	**Python Script** reads both R Shiny data and the **Excel** file.
4. Quickbooks & Data Ingestion	The Python API Script takes the calculated final price and performs two actions: 1) Creates the official **QuickBooks Estimate** via API. 2) Pushes the full quote record to the central BigQuery Data Warehouse.	**Python API Script** integrates **QuickBooks API** and **BigQuery Connector**.
5. Final Output & Reporting	The final price is returned to the R Shiny UI for staff review, and the customer receives the official QuickBooks Estimate PDF.	**R Shiny UI** (Review) and **QuickBooks Estimate PDF** (Output).

2. Fixing the Proposal & Enhancing the Workflow

The core focus of these fixes is mitigating the high risk introduced by using an Excel file for core pricing data, while maximizing the **Ease of Use** of the R Shiny interface.

Proposal Component	Fixes Requested	Critical/Enhancement Fix Applied
R Shiny dashboard (Customer Info)	*Fix:* GreenTech color palette/style. *Fix:* No technical training (**Ease of Use**).	The R Shiny UI handles input, triggering Python, showing results, and confirming the QuickBooks push.
Central Excel Sheet (Supplier Info)	**CRITICAL FIX:** More reliable storage (Only one place). *Fix:* Little technical training. IT department may need to be familiar to make future changes.	Rigorous data validation and Excel parsing logi**c** in the Python script to catch human errors, data formatting issues, or missing cells *before* calculation.
Python Script (Calculation)	*Fix:* Preserve accuracy. *Fix:* Reduces quoting time.	The **R Shiny trigger** guarantees **Ease of Use** by removing manual file execution. Python can correctly handle the complex task of **reading formulas/data ranges** from the dynamic Excel file.
Python API Script (QuickBooks)	*Fix:* Reduce quoting time.	Build robust error handling into the API and return clear, non-technical messages to the R Shiny UI if the QuickBooks push fails, maintaining a smooth user experience.
BigQuery (Sales/Customers/Quotes)	(No fixes requested)	BI analysis is now separated from the pricing data source. The BigQuery schema must be simplified to focus only on capturing the final sales, quote, and customer information for analytics.

3. Custom Solution vs. Low-Code Alternatives

This comparison highlights why the custom stack, even with the **Excel constraint**, is superior for meeting the non-negotiable **Key Requirements** (Preserve Accuracy, Scalability).

Requirement (Student Brief)	Custom Solution (R Shiny/Python/BigQuery)	Low-Code/No-Code System	Strategic Advantage
Preserve Accuracy	**High:** Proprietary formulas are coded in Python, protected, and fully customizable, ensuring high fidelity despite the Excel source.	**Low:** Formulas must be rebuilt in a limited visual editor, risking errors and exposing IP.	**Custom Stack** (Python's control over logic)
Reliability/Auditability	**Low:** The Excel file is the single point of failure and inaccuracy. Python mitigation reduces the risk.	**Medium:** Low-code systems generally use a secure, version-controlled database by default, avoiding the Excel risk entirely.	**Low-Code System** (Better core data storage)
Brand Consistency	**High:** R Shiny allows full HTML/CSS customization to strictly follow the GreenTech style guide.	**Low:** Limited to vendor's restrictive themes; difficult to implement required brand guidelines.	**Custom Stack** (R Shiny UI)
Scalability for Franchisees	**High:** Python/BigQuery is inherently scalable. The core challenge is training new franchisees on the risk management of the Excel file.	**Medium:** Scalability is tied to vendor pricing tiers. Data is safe, but cost increases with franchise growth.	**Custom Stack** (Cost-efficient growth)
Offline Access	**Requires RShiny Dashboard:** RShiny Dashboard must integrate a mobile-first solution for capture and later sync to the R Shiny/Python processing stack.	**Varies Widely:** Functionality often exists, but requires the most expensive licensing tier.	**Neutral/Depends on RShiny Dashboard solution**
Ease of Use	**High:** The single R Shiny interface to input data and trigger calculation provides a seamless, non-technical experience for staff.	**Medium:** Staff may need to switch between the quoting UI and the QuickBooks interface.	**Custom Stack** (Single UI)